Google TPU 微架构与软件开发平台深入综述

从脉动阵列到光互连超算系统的架构演进与设计理念

技术调研报告　Claude (AI Assistant)

2026 年 4 月

摘要

领域专用架构（Domain-Specific Architecture, DSA）已成为当代人工智能基础设施的核心支柱。Google Tensor Processing Unit（TPU）作为业界首个大规模部署的神经网络加速器，自 2016 年问世以来经历了七代演进，其设计实践为 AI 芯片架构研究提供了宝贵的工程经验与理论启示。本综述基于 Google 官方发布的核心技术文献（Jouppi et al., ISCA 2017; Jouppi et al., ISCA 2023; Norrie et al., IEEE Micro 2021），系统性地梳理 TPU 从第一代推理加速器到第七代 Ironwood 的架构演进脉络，深入剖析其微架构设计决策背后的工程权衡与技术考量。

本文首先回顾 TPU v1 的设计背景与核心创新，详细分析 65536 单元脉动阵列的架构特征及其对神经网络推理工作负载的适配机制。随后，本文追踪 TPU 从推理专用到训推兼顾的设计范式转变，深入讨论 TPU v2/v3 引入的高带宽内存、芯片间互连等关键技术的工程实现细节。本文的核心章节聚焦于 TPU v4 的光互连架构革命，系统阐述光路交换（OCS）技术的系统级价值、SparseCore 的嵌入式加速机制，以及 4096 芯片超算系统的拓扑设计原理。此外，本文还纳入了对 2024–2025 年最新代际 Trillium（v6）与 Ironwood（v7）的分析，补充了大规模部署案例与经济性评估。

在软件栈分析部分，本文深入探讨 XLA 编译器的优化策略、JAX/Pathways 与 TPU 硬件的协同设计机制，以及 Google 在编译器-硬件协同优化方面的实践经验。通过与 NVIDIA GPU 架构的系统性对比和专利布局分析，本文揭示 DSA 与通用 GPU 在设计哲学、编程模型、生态策略和知识产权策略上的本质差异。最后，本文以批判性视角审视 TPU 成就背后的资料不对称、封闭生态权衡与能效归因问题。

关键词： Google TPU、脉动阵列、光互连、领域专用架构、XLA 编译器、超算系统、神经网络加速器、Trillium、Ironwood、专利布局

目录

1 引言：神经网络加速的架构需求 **3**
1.1 深度学习时代的计算挑战 . . . 3
1.2 领域专用架构的兴起 . . . 3
1.3 TPU 的设计哲学与演进历程 . . . 4
1.4 本综述的研究范围与贡献 . . . 4

2 TPU v1：脉动阵列的首次大规模实践 **5**
2.1 设计背景与工程约束 . . . 5
2.2 脉动阵列架构的工程实现 . . . 6
2.3 确定性执行模型与延迟保证 . . . 7
2.4 TPU v1 的局限性与设计教训 . . . 8

3 TPU v2 与 v3：向大规模训练扩展 **8**
3.1 从推理到训练的架构挑战 . . . 8
3.2 设计约束与“快速上市”策略 . . . 9
3.3 高带宽内存（HBM）的引入 . . . 9
3.4 芯片间互连（ICI）架构 . . . 10
3.5 bfloat16：精度与效率的平衡 . . . 11
3.6 软件栈的演进 . . . 11
3.7 性能评估与部署经验 . . . 12

4 TPU v4：光互连超算系统的架构革命 **12**
4.1 设计目标与挑战 . . . 12
4.2 TPU v4i：推理专用变体与十条设计教训 . . . 14
4.3 光路交换（OCS）技术原理 . . . 15
4.4 动态拓扑与 3D 环面 . . . 15
4.5 SparseCore：嵌入式专用加速 . . . 18
4.6 4096 芯片超算系统 . . . 19
4.7 性能对比与能效分析 . . . 19

5 TPU v6 (Trillium) 与 v7 (Ironwood)：推理时代的回归与放大 **21**
5.1 Trillium：训练与推理的双重飞跃 . . . 21
5.2 Ironwood：专为推理而生的第七代 TPU . . . 21
5.3 规模部署与 Jupiter 网络 . . . 22

6 软件栈、专利布局与横向对比 **22**
6.1 XLA 编译器架构 . . . 22
6.2 脉动阵列代码生成 . . . 23
6.3 分布式并行优化 . . . 23

6.4 JAX 与函数式编程模型 24
6.5 Pathways 运行时与超大规模分布式执行 24
6.6 软件生态的比较分析 25
6.7 专利布局：开放论文与收敛专利的反差 25
6.8 与 NVIDIA GPU 的架构对比 26
6.9 MLPerf 基准测试分析 26
6.10 TCO 与部署成本分析 27
6.11 性能瓶颈与优化策略 28

7 批判性讨论：成就、代价与未解张力 28
7.1 资料不对称与方法论的局限 28
7.2 封闭生态的长短期权衡 28
7.3 脉动阵列与稀疏性、动态性的张力 29
7.4 能效宣称的归因问题 29

8 未来趋势与开放问题 29
8.1 从训练到推理的范式转变 29
8.2 稀疏性与低精度计算 30
8.3 存算一体与近内存计算 30
8.4 软件生态的开放与标准化 30
8.5 可持续性与绿色计算 31

9 结论 31

1 引言：神经网络加速的架构需求

1.1 深度学习时代的计算挑战

深度神经网络的崛起对计算系统提出了前所未有的挑战。自 2012 年 AlexNet 在 ImageNet 竞赛中取得突破性成果以来，神经网络模型的规模和复杂度呈现出指数级增长态势。这种增长不仅体现在模型参数量上——从 AlexNet 的 6000 万参数到 GPT-3 的 1750 亿参数，再到如今万亿参数级别的超大规模语言模型——更体现在训练这些模型所需的计算量上。根据 OpenAI 的估算，自 2012 年以来，最大规模 AI 训练运行的计算量增长了超过 30 万倍，远超摩尔定律所预测的硬件性能提升速度。

这种计算需求的爆发式增长暴露了通用处理器架构的根本性局限。中央处理器（CPU）虽然在通用计算任务上表现出色，但其面向顺序执行优化的设计哲学与神经网络高度并行、计算密集的特征存在结构性错配。图形处理器（GPU）凭借其大规模并行计算能力，在一定程度上缓解了这一问题，但 GPU 本质上仍为图形渲染等通用并行任务而设计，其单指令多线程（SIMT）执行模型、复杂的缓存层次结构和通用编程接口，在神经网络计算场景下仍存在显著的效率损失。

具体而言，神经网络推理和训练的核心计算模式——矩阵乘法（GEMM）——在 GPU 上执行时面临多重开销。首先是控制开销：GPU 需要将矩阵运算分解为大量独立指令，每条指令都需经过取指、译码、发射等流水线阶段，这些控制逻辑消耗了可观的芯片面积和功耗。其次是内存访问开销：GPU 的缓存层次结构虽然对通用数据访问模式有效，但对于神经网络中规则的数据流模式而言，复杂的缓存管理反而成为负担。此外，GPU 的动态调度机制虽然提高了通用性，但引入了执行时间的不确定性，这对于需要严格延迟保证的数据中心服务而言是一个重要缺陷。

1.2 领域专用架构的兴起

面对通用处理器架构的局限性，计算机架构学界和工业界开始重新审视领域专用架构（Domain-Specific Architecture, DSA）的价值。DSA 的设计理念是：通过针对特定应用领域的计算特征进行深度定制，在牺牲通用性的前提下，换取数量级的能效比和性能提升。这一理念并非全新——数字信号处理器（DSP）、网络处理器等专用芯片早已存在——但在深度学习时代，DSA 获得了前所未有的关注和投入。

Google 于 2013 年启动的 TPU 项目是 DSA 理念在深度学习领域的首次大规模实践。与渐进式的架构优化不同，TPU 采用了革命性的设计思路：完全围绕神经网络的核心计算模式——矩阵乘法——构建专门的硬件加速引擎。这种设计思路的核心是脉动阵列（Systolic Array）架构，一种由 H. T. Kung 等人在 1980 年代提出的数据流计算范式。脉动阵列的关键特征在于其规则的数据流动模式：数据以节律化的方式在计算单元阵列中流动，每个单元在数据经过时执行简单的乘加操作，计算结果传递给相邻单元。这种模式最大限度地减少了内存访问，将计算密度推向了理论极限。

1.3 TPU 的设计哲学与演进历程

TPU 的设计体现了 Google 在计算机系统领域的深厚积累和对机器学习工作负载的深刻理解。从 2016 年发布的第一代 TPU 到 2025 年的第七代 Ironwood，TPU 的演进历程可以划分为三个主要阶段。

第一阶段（2016-2017 年）以 TPU v1 为代表，标志着 Google 对神经网络推理加速的首次探索。这一代 TPU 采用了纯粹的脉动阵列设计，针对数据中心推理场景的延迟和能效需求进行了深度优化。TPU v1 的设计决策——如选择 INT8 精度、采用软件管理的片上内存、去除复杂的缓存和分支预测逻辑——都体现了对特定应用场景的极致特化。

第二阶段（2017-2021 年）见证了 TPU 从推理专用向训推兼顾的转变。TPU v2 和 v3 在保持脉动阵列核心计算引擎的同时，引入了高带宽内存（HBM）、芯片间互连（ICI）等关键技术，将加速器的适用范围扩展到大规模分布式训练。这一阶段的设计重点在于如何在保持高能效的同时，满足训练工作负载对内存容量、带宽和扩展性的苛刻需求。

第三阶段（2021 年至今）以 TPU v4 为里程碑，代表了 AI 加速器架构的重大飞跃。TPU v4 引入了革命性的光路交换（Optical Circuit Switching, OCS）技术，使得超算系统的互连拓扑可以动态重构。这一创新不仅解决了大规模集群的可扩展性和可靠性问题，更为未来的自适应互连架构指明了方向。

1.4 本综述的研究范围与贡献

本综述聚焦于 Google TPU 的架构演进历程与设计理念，旨在为计算机架构研究者和 AI 系统工程师提供一份系统性的技术参考。与现有的技术博客和新闻报道不同，本综述基于 Google 官方发布的核心学术文献（包括 ISCA 2017、ISCA 2023 等顶级会议论文和 IEEE Micro 综述文章），深入剖析 TPU 微架构设计的工程权衡和技术细节。

本文的主要贡献包括四个方面。首先，本文系统梳理了 TPU 从 v1 到 v7 的架构演进脉络，揭示了每一代设计决策背后的技术动因和约束条件。其次，本文深入分析了 TPU 的核心技术创新——包括脉动阵列的微观架构、光互连的系统级设计、SparseCore 的专用加速机制——并评估这些创新对整体系统性能的影响。第三，本文补充了软件栈、专利布局与横向对比分析，揭示 DSA 与通用 GPU 在知识产权策略和生态系统上的本质差异。最后，本文以批判性视角审视厂商宣称与独立验证之间的鸿沟，以及封闭生态的长短期权衡。

在组织结构上，本文第二章详细分析 TPU v1 的设计背景与架构特征；第三章追踪 TPU v2/v3 向训练场景的扩展；第四章深入探讨 TPU v4 的光互连超算系统；第五章分析最新的 Trillium（v6）与 Ironwood（v7）；第六章剖析软件栈、专利布局与横向对比；第七章进行批判性讨论；第八章展望未来趋势与开放问题。通过这一系统性的技术梳理，本文期望为 AI 加速器架构研究提供有价值的参考和启示。

2 TPU v1：脉动阵列的首次大规模实践

2.1 设计背景与工程约束

TPU v1 的设计始于 2013 年，当时 Google 的数据中心已经开始大规模部署深度神经网络，用于支持搜索排名、语音识别、图像分类、机器翻译等核心产品功能。根据 Google 当时的内部估算，如果每个用户每天进行三分钟的语音搜索，就需要将数据中心的计算能力扩大一倍。这一计算需求的增长速度与通用处理器性能提升速度之间的鸿沟，促使 Google 决定自主开发专用加速器。

在设计 TPU v1 时，Google 的工程团队面临一系列严格的约束条件。首先是应用场景的限定：TPU v1 专门针对神经网络推理（inference）阶段进行优化，而非训练（training）。这一决策基于对 Google 数据中心工作负载的深入分析：推理计算在 2013 年已经占据了数据中心 AI 计算的绝大部分，且对延迟和能效有着更严格的要求。其次是部署环境的约束：TPU v1 需要无缝集成到 Google 现有的数据中心基础设施中，这意味着它必须兼容标准的服务器形态、散热设计和电源配置。

更为关键的是，TPU v1 的设计必须在性能、功耗、延迟和成本之间进行精细的工程权衡。根据 Jouppi 等人的分析（ISCA 2017），数据中心中的神经网络推理工作负载具有三个关键特征：计算密集性（大量的矩阵乘加操作）、规则的数据访问模式（权重和激活值的有序访问），以及对延迟尾部的严格要求（99 百分位响应时间）。这些特征决定了 TPU v1 的设计必须优先考虑确定性执行、高计算密度和极低的内存访问开销。

2.2 脉动阵列架构的工程实现

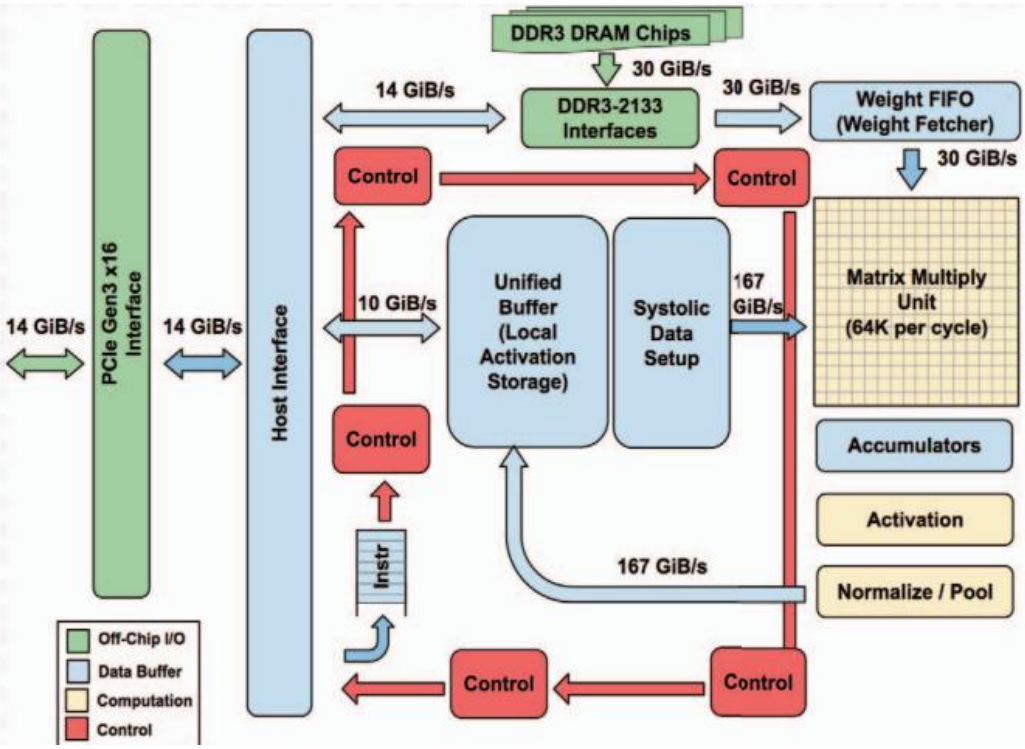

图 1: TPU v1 架构框图, 展示了脉动阵列 (Matrix Multiply Unit)、统一缓冲区 (Unified Buffer)、权重 FIFO 等核心组件及其数据流。来源：Jouppi et al., ISCA 2017

TPU v1 的核心创新在于其大规模脉动阵列的实现。该阵列由 65536 个 8 位整数乘加单元组成，以 256×256 的二维网格排列。每个乘加单元（Multiply-Accumulate Unit, MAC）能够在每个时钟周期执行一次 8 位乘法操作，并将结果累加到本地累加器中。这种粗粒度的计算单元组织方式，与 GPU 中细粒度的 SIMT 执行模型形成了鲜明对比。

脉动阵列的工作模式遵循严格的数据流规范。在矩阵乘法运算中，权重矩阵被预先加载到阵列的各个单元中，激活向量从左侧逐列流入，部分和向量从上侧流入。每个单元在其本地权重与流入的激活值相乘后，将结果加到流入的部分和上，然后将新的部分和传递给下方的单元。经过 256 个时钟周期的流动，结果矩阵的各行从阵列底部依次流出。这种数据流动模式的核心优势在于其极致的内存带宽效率：一旦权重被加载，整个矩阵乘法过程仅需从内存读取激活向量和写回结果向量，中间计算完全不依赖外部内存访问。

TPU v1 的脉动阵列实现采用了多项关键的技术优化。首先是精度选择：TPU v1 专注于 8 位整数（INT8）运算，这一决策基于对神经网络推理精度的深入分析。Google 的研究表明，对于大多数生产环境的神经网络模型，8 位量化可以在保持可接受准确率的同时，将计算密度和能效提升四倍。其次是累加精度：虽然乘法是 8 位的，但累加操作使用 32 位整数，以防止溢出并保持数值稳定性。

在物理实现层面，TPU v1 采用了 28 纳米工艺制造，芯片面积约 331 平方毫米，功耗控制在 75 瓦以内。芯片除了脉动阵列外，还包括 24 兆字节的片上 SRAM、PCIe Gen3

x8 接口、以及用于数据和指令管理的控制逻辑。值得注意的是，TPU v1 的片上内存采用软件管理而非硬件缓存，这进一步简化了硬件设计并消除了缓存未命中的不确定性。

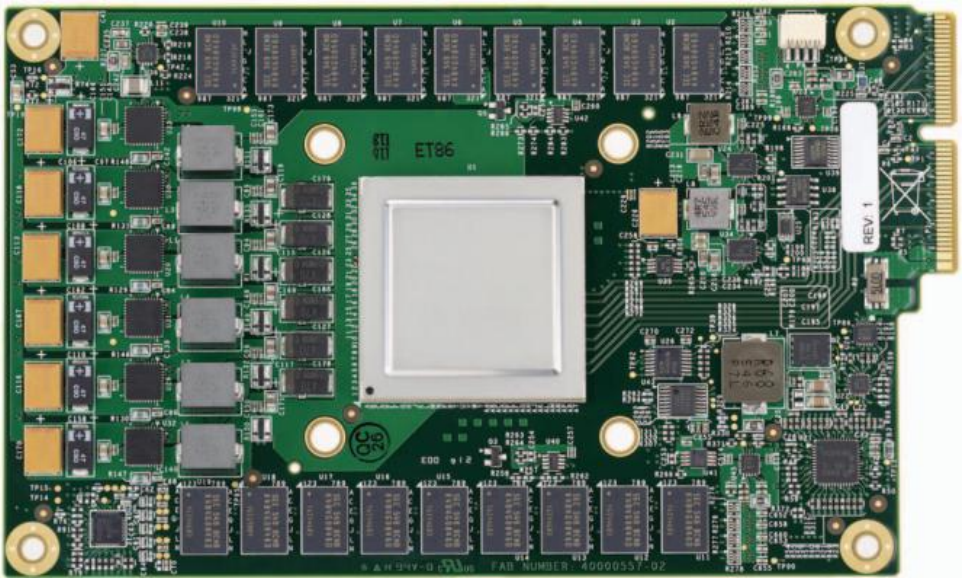

图 2: TPU v1 加速卡实物照片，展示了 PCIe 接口、散热片和周围的 DRAM 芯片。来源：Jouppi et al., ISCA 2017

2.3 确定性执行模型与延迟保证

TPU v1 的另一个关键设计特征是其确定性的执行模型。与 CPU 和 GPU 的动态调度、推测执行、乱序执行等复杂机制不同，TPU v1 的执行流程完全由编译时确定的指令序列控制，没有任何运行时调度开销或执行时间的变异性。这种确定性源于多个层面的设计决策。

在指令集架构层面，TPU v1 采用了一种简化但高效的原语集。核心指令包括：权重加载指令（用于将权重矩阵预加载到脉动阵列）、矩阵乘加指令（触发激活向量流入阵列并启动计算）、激活函数指令（对阵列输出应用 ReLU、sigmoid 等非线性函数）、以及数据移动指令（在片上 SRAM 和主机内存之间传输数据）。所有指令的执行时间都是固定的、可预测的，这使得编译器可以精确计算任意计算图的总执行时间。

在内存系统层面，TPU v1 的软件管理 SRAM 消除了缓存未命中的不确定性。编译器在编译阶段就确定了所有数据的内存布局，生成的代码显式控制数据的加载和存储时机。这种显式内存管理虽然增加了编译器的复杂性，但为应用程序提供了严格的延迟保证。对于需要满足 99 百分位延迟要求的数据中心服务而言，这种确定性至关重要。

根据 Jouppi 等人在 ISCA 2017 论文中的测量数据，TPU v1 在典型神经网络推理工作负载上展现出显著的性能和能效优势。如表 3所示，与同期的 Intel Haswell CPU 和 NVIDIA K80 GPU 相比，TPU v1 在性能、能效和计算密度方面均有显著优势。

Model	Die										Benchmarked Servers				
	mm^2	nm	MHz	TDP	Measured		TOPS/s		GB/s	On-Chip Memory	Dies	DRAM Size	TDP	Measured	
					Idle	Busy	8b	FP						Idle	Busy
Haswell E5-2699 v3	662	22	2300	145W	41W	145W	2.6	1.3	51	51 MiB	2	256 GiB	504W	159W	455W
NVIDIA K80 (2 dies/card)	561	28	560	150W	25W	98W	--	2.8	160	8 MiB	8	256 GiB (host) + 12 GiB x 8	1838W	357W	991W
TPU	<331*	28	700	75W	28W	40W	92	--	34	28 MiB	4	256 GiB (host) + 8 GiB x 4	861W	290W	384W

图 3: TPU v1 与 Intel Haswell CPU、NVIDIA K80 GPU 的规格对比表。来源：Jouppi et al., ISCA 2017

与 Intel Haswell CPU 相比，TPU v1 的平均推理速度快 15 至 30 倍，能效比（每瓦特性能）高出 30 至 80 倍。与 NVIDIA K80 GPU 相比，TPU v1 的速度优势为 15 至 30 倍，能效优势为 30 至 80 倍。这些数字证明了 DSA 设计范式在特定应用场景下的巨大潜力。

2.4 TPU v1 的局限性与设计教训

尽管 TPU v1 在推理场景下表现出色，但其设计也存在明显的局限性，这些局限性为后续代的架构演进提供了重要的设计教训。首先是精度的局限：TPU v1 仅支持 8 位整数运算，虽然这对于大多数推理场景已经足够，但对于需要更高数值精度的应用（如某些科学计算任务或需要保留更多信息的中间层）则显得力不从心。

其次是内存容量的瓶颈：24MB 的片上 SRAM 虽然对于当时的模型已经足够，但随着神经网络模型规模的快速增长，这一容量很快成为限制因素。此外，TPU v1 与主机 CPU 之间的 PCIe 接口带宽也成为大规模数据传输时的瓶颈。这些内存和 IO 限制使得 TPU v1 难以应对更大规模的模型和批处理场景。

最为根本的局限性在于 TPU v1 的设计完全针对推理优化，无法有效支持训练工作负载。神经网络训练涉及反向传播算法，需要前向传播、损失计算、梯度反向传播和权重更新等多个阶段的复杂协调。训练过程还需要更高的数值精度（通常需要 16 位或 32 位浮点）来保持数值稳定性，以及更大的内存容量来存储中间激活值和梯度。这些需求超出了 TPU v1 的设计范围。

这些局限性促使 Google 在 TPU v1 部署后不久就开始规划下一代架构，目标是设计一款既能高效支持推理，又能胜任训练的通用神经网络加速器。这一设计目标的扩展，开启了 TPU 架构演进的新篇章。

3 TPU v2 与 v3：向大规模训练扩展

3.1 从推理到训练的架构挑战

将 TPU 从推理专用加速器扩展为支持大规模训练的通用平台，是一项充满挑战的工程任务。神经网络训练与推理在计算特征、资源需求和系统架构上存在根本性差异。理解这些差异，是把握 TPU v2/v3 设计决策的关键。

在计算特征层面，训练涉及反向传播算法，这是一个包含前向传播、损失计算、梯度反向传播和权重更新四个阶段的复杂过程。前向传播与推理类似，计算网络各层的激活值；但反向传播需要计算每一层权重的梯度，这涉及额外的矩阵运算和大量的中间数据传递。此外，训练通常采用随机梯度下降（SGD）或其变种作为优化算法，需要维护动量、自适应学习率等优化器状态，进一步增加了内存需求。

在数值精度层面，训练对数值稳定性有更高要求。推理通常可以使用低精度量化（如 8 位整数），但训练过程中的梯度计算涉及大量小数值的累加，低精度容易导致数值下溢或精度损失。Google 的研究表明，16 位浮点格式（bfloat16）是训练场景下精度与效率的良好平衡点，它保留了 32 位浮点的动态范围（8 位指数），但将尾数减少到 7 位，从而在保持数值稳定性的同时，将计算和存储需求减半。

在系统架构层面，大规模训练需要多芯片协同工作。现代神经网络模型通常包含数十亿甚至数千亿参数，单芯片的内存容量和计算能力难以满足训练需求。因此，训练加速器必须支持高效的多芯片扩展，包括芯片间的高速通信、分布式参数同步、以及容错和弹性扩展机制。这些需求对芯片的互连带宽、拓扑结构和系统软件都提出了严苛要求。

3.2 设计约束与“快速上市”策略

TPU v2 和 v3 的设计面临与 v1 截然不同的约束。Norrie 等人 [4] 回顾道，设计团队将目标明确分为“必须做好” (first bucket) 和“足够好” (second bucket) 两档：

- 必须做好：快速构建、高性能、可扩展、对新负载开箱即用、成本可控；
- 足够好：芯片面积效率、双核共享 HBM 的优雅性、版图利用率等。

这一策略反映了实际工程中的时间与人员约束。例如，为了加速设计进度，团队故意简化了双核共享 HBM 的控制器，容忍了不够高效的芯片版图，并使用 FIFO 来简化 XLA 编译器的调度。这种“先上市后优化”的工程哲学，与学术研究中常见的“追求最优解”形成了有趣的对照。

3.3 高带宽内存（HBM）的引入

TPU v2/v3 采用了双核心设计，每个芯片包含两个独立的计算核心，通过片上互连路由器（Interconnect Router）实现高速通信。如图 4所示，每个核心包含矩阵乘法单元（Matrix Multiply Unit）、向量单元（Vector Unit）、标量单元（Scalar Unit）和转置/置换单元（Transpose/Permute Unit），形成了完整的计算流水线。

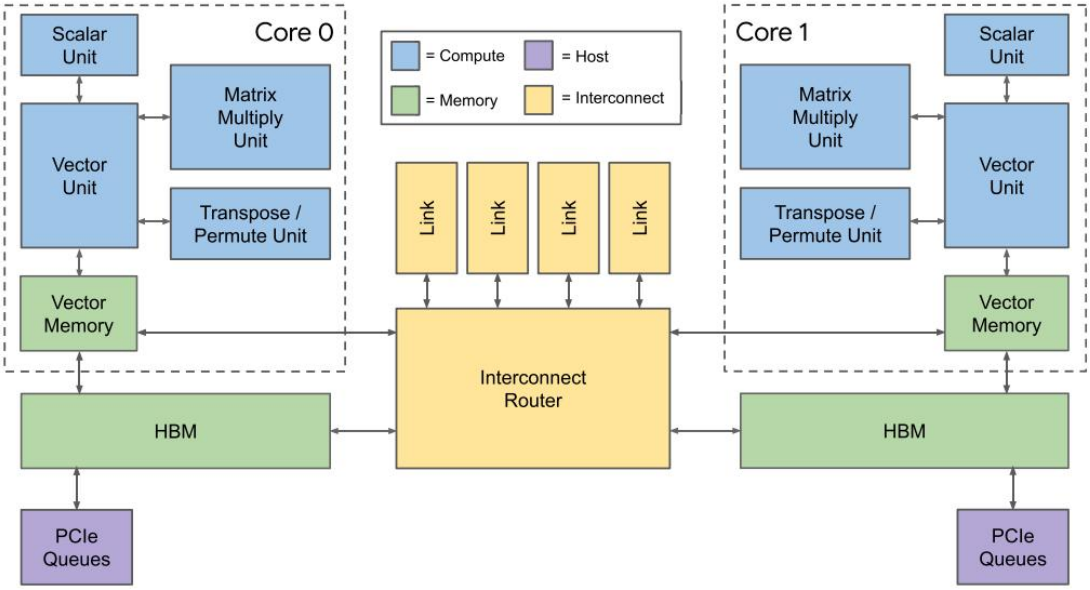

图 4: TPU v2/v3 双核心架构框图，展示了计算单元、向量内存、HBM 和互连路由器的组织方式。来源：Norrie et al., IEEE Micro 2021

TPU v2 最显著的架构改进之一是引入了高带宽内存（High Bandwidth Memory, HBM）。HBM 是一种 3D 堆叠内存技术，通过硅通孔（Through-Silicon Via, TSV）将多个 DRAM 芯片垂直堆叠在基础逻辑芯片上，从而实现极高的内存带宽和密度。

在 TPU v2 中，每个芯片集成了 64GB 的 HBM2，提供约 600-700 GB/s 的内存带宽。与 TPU v1 的 24MB 片上 SRAM 相比，这是一个数量级的提升。HBM 的引入解决了训练场景下的内存容量瓶颈，使得 TPU 可以容纳更大规模的模型和更大的批处理大小。更重要的是，HBM 的高带宽特性与脉动阵列的高计算吞吐相匹配，避免了计算单元因等待数据而空闲的情况。

HBM 的物理实现也带来了新的工程挑战。首先是散热问题：HBM 的 3D 堆叠结构增加了热阻，需要更精细的散热设计。其次是制造成本：HBM 的封装复杂度远高于传统 GDDR 内存，这直接反映在芯片成本上。Google 的解决方案是采用双芯片封装（Dual-Chip Module），每个封装包含两个 TPU v2 芯片，共享散热和电源设计，从而在一定程序上摊薄了成本。

TPU v3 进一步升级了 HBM 配置，将容量增加到 128GB，带宽提升到 900 GB/s。这一升级反映了神经网络模型规模的持续增长：从 2017 年到 2019 年，最先进的模型参数量增长了数十倍，对内存容量和带宽的需求也随之水涨船高。TPU v3 还引入了液冷技术，以应对 HBM 和计算核心产生的高热流密度，支持更高功率密度的部署。

3.4 芯片间互连（ICI）架构

大规模分布式训练的核心挑战之一是如何高效地在数百甚至数千个加速器之间传递数据和同步状态。TPU v2/v3 引入了专用的芯片间互连（Inter-Chip Interconnect, ICI），这是一个针对 AI 训练工作负载定制的高速通信网络。

ICI 的设计目标是在保持高带宽的同时，最小化通信延迟和系统复杂度。与通用的

InfiniBand 或以太网相比，ICI 针对神经网络训练的通信模式进行了深度优化。在数据并行训练中，每个芯片处理不同的数据批次，但需要频繁同步梯度（All-Reduce 操作）；在模型并行训练中，不同芯片负责网络的不同层，需要传递层间的激活值和梯度。ICI 的拓扑设计和路由算法针对这些模式进行了优化。

TPU v2 支持构建最多 256 个芯片的 Pod 配置，芯片之间通过高速链路连接成 2D 环面（Torus）拓扑。在这种拓扑中，每个芯片与四个相邻芯片直接相连，数据可以通过最短路径在多跳之间传递。TPU v3 将 Pod 规模扩展到 1024 个芯片，并优化了路由算法以降低多跳通信的延迟。

ICI 的物理实现采用了高速串行链路技术，单链路带宽达到数百 Gbps。与光互连相比，这一代的 ICI 仍采用电信号传输，但 Google 通过精心的信号完整性设计和先进的封装技术，实现了高带宽密度和低功耗。每个 TPU 芯片集成了多个 ICI 端口，支持同时与多个邻居通信，从而实现高效的集合通信操作。

3.5 bfloat16：精度与效率的平衡

TPU v2/v3 引入的 bfloat16（Brain Floating Point 16）格式，是神经网络加速器设计中的一个重要创新。这一格式由 Google Brain 团队开发，旨在为神经网络训练提供精度与效率的最佳平衡。

bfloat16 的位布局与 IEEE 754 标准的 16 位浮点（half-precision 或 FP16）有显著不同。标准的 FP16 使用 5 位指数和 10 位尾数，而 bfloat16 使用 8 位指数和 7 位尾数。这意味着 bfloat16 具有与 32 位单精度浮点（FP32）相同的动态范围（可以表示同样大或同样小的数值），但尾数精度较低。对于神经网络训练而言，动态范围的重要性通常高于尾数精度，因为梯度值可能跨越多个数量级，而一定程度的精度损失对训练收敛的影响相对较小。

在硬件实现层面，bfloat16 的简化为乘加单元的设计带来了显著优势。由于尾数位数从 23 位（FP32）减少到 7 位，乘法器的复杂度大幅降低，可以在同样的芯片面积内容纳更多的计算单元。同时，由于指数格式与 FP32 一致，bfloat16 与 FP32 之间的转换非常简单，不需要复杂的规范化或舍入逻辑。

Google 的研究表明，在大多数神经网络训练任务上，bfloat16 可以达到与 FP32 相当的收敛速度和最终模型质量，同时提供接近 2 倍的计算吞吐提升。这一发现对于大规模训练的经济性具有重要意义：在 TPU v2 上采用 bfloat16 训练，可以在保持模型质量的前提下，将训练时间和成本减半。

3.6 软件栈的演进

TPU v2/v3 的硬件扩展需要相应的软件栈支持。Google 围绕 TPU 构建了一套完整的软件生态系统，包括编译器、运行时库和高级框架集成。

XLA（Accelerated Linear Algebra）编译器是这一软件栈的核心。XLA 接受来自 TensorFlow、JAX 等框架的计算图表示，执行一系列优化变换，生成针对 TPU 优化的机器码。对于 TPU v2/v3，XLA 的优化重点包括：将大规模计算图划分为可在多芯片

上并行执行的子图；优化张量在 HBM 和片上 SRAM 之间的数据布局，最大化数据复用；以及生成高效的 ICI 通信代码，最小化芯片间同步开销。

在分布式训练支持方面，Google 开发了多种并行策略。数据并行（Data Parallelism）是最基础的策略，将不同数据批次分配给不同芯片，每轮迭代后通过 All-Reduce 同步梯度。模型并行（Model Parallelism）将网络的不同层分配给不同芯片，适用于单层参数量超过单芯片内存容量的情况。流水线并行（Pipeline Parallelism）将网络划分为多个阶段，每个阶段由一个或多个芯片负责，通过流水线方式重叠计算和通信。XLA 编译器可以自动或半自动地选择和应用这些并行策略，减轻开发者的优化负担。

3.7 性能评估与部署经验

根据 Google 在 IEEE Micro 2021（Norrie et al.）和 ISCA 2023（Jouppi et al.）论文中的报告，TPU v2/v3 在内部工作负载上展现出显著的性能优势。在 Google 的数据中心，TPU v2/v3 支撑的机器学习工作负载包括搜索排名、广告推荐、语音识别、自然语言处理、计算机视觉等多个领域。

在系统级效率方面，TPU v2/v3 的 Pod 配置在大规模分布式训练上展现出良好的扩展性。根据测量数据，在 1024 芯片的 TPU v3 Pod 上训练大型 Transformer 模型，可以实现约 80-90% 的线性扩展效率。这意味着相比单芯片，1024 芯片的训练速度提升约 800-900 倍，接近理想的线性扩展。这一高效率得益于 ICI 互连的高带宽低延迟设计，以及 XLA 编译器对通信和计算的有效重叠。

然而，TPU v2/v3 的设计也面临一些挑战和局限性。首先是通用性不足：虽然相比 TPU v1 已经扩展了适用范围，但 TPU v2/v3 仍然是专门针对神经网络优化的加速器，对于其他类型的计算任务（如稀疏线性代数、图计算等）支持有限。其次是生态系统的封闭性：TPU 只能在 Google Cloud Platform 上使用，且主要支持 TensorFlow 和 JAX 框架，对于使用 PyTorch 等其他框架的开发者而言存在迁移成本。

这些经验教训为 TPU v4 的设计提供了重要参考。下一代 TPU 需要在保持高能效的同时，进一步提升可扩展性、灵活性和易用性，以应对快速发展的机器学习模型和工作负载需求。

4 TPU v4：光互连超算系统的架构革命

4.1 设计目标与挑战

TPU v4 代表了 Google 在 AI 加速器架构上的重大突破，其核心创新在于引入了光路交换（Optical Circuit Switching, OCS）技术，使得超算系统的互连拓扑可以动态重构。这一创新的驱动力源于对大规模分布式训练需求的深刻理解和工程预判。如表 5所示，TPU v4 在峰值算力、内存带宽、工艺制程和扩展规模方面相比 TPU v3 均有显著提升。

	Google TPUv4	TPUv3
Production deployment	2020	2018
Peak TFLOPS	275 (bf16 or int8)	123 (bf16)
Clock Rate	1050 MHz	940 MHz
Tech. node, Die size	7 nm, <600 mm2	16 nm, < 700 mm2
Transistor count	22 billion	10 billion
Chips per CPU host	4	8
TDP	N.A.	N.A.
Idle, min/mean/max power	90, 121/170/192 W	123, 175/220/262 W
Inter Chip Interconnect	6 links @ 50 GB/s	4 links @ 70 GB/s
Largest scale configuration	4096 chips	1024 chips
Processor Style	Single Instruction 2D Data	Single Instruction 2D Data
Processors / Chip	2	2
Threads / Core	1	1
SparseCores / Chip	4	2
On Chip Memory	128 (CMEM) + 32 MiB (VMEM) + 10 MiB (spMEM)	32 MiB (VMEM) + 5 MiB (spMEM)
Register File Size	0.25 MiB	0.25 MiB
HBM2 capacity, BW	32 GiB, 1200 GB/s	32 GiB, 900 GB/s

图 5: TPU v4 与 TPU v3 规格对比表，展示了峰值算力、制程工艺、内存带宽和最大规模配置等关键参数。来源：Jouppi et al., ISCA 2023

随着 Transformer 类大语言模型的崛起，神经网络训练的规模需求呈现出超指数增长态势。从 BERT（3.4 亿参数）到 GPT-3（1750 亿参数）再到 PaLM（5400 亿参数），模型规模的每一次跃升都对计算基础设施提出了数量级更高的需求。在这种背景下，固定拓扑的传统互连架构面临着根本性挑战：如何在保证高性能的同时，实现足够的灵活性、可靠性和可维护性?

Google 的工程团队在设计 TPU v4 时，设定了一系列雄心勃勃的目标。首先是规模目标：支持 4096 个芯片的超算系统，相比 TPU v3 的 1024 芯片配置实现 4 倍扩展。其次是灵活性目标：允许根据工作负载需求动态调整互连拓扑，优化通信性能。第三是可靠性目标：实现容错和弹性扩展，确保单点故障不会导致整个系统停机。第四是部署目标：简化大规模系统的部署和运维复杂度。

传统的电互连技术难以同时满足这些目标。基于铜缆的高速电信号传输在带宽密

度、传输距离和功耗方面都存在物理极限。虽然 InfiniBand 等高性能互连技术可以提供高带宽，但其固定拓扑和复杂的布线使得大规模部署和动态重构极为困难。Google 的解决方案是采用光互连技术，利用光信号的高带宽、低损耗和可重构特性，构建下一代超算系统。

4.2 TPU v4i：推理专用变体与十条设计教训

TPU v4 的推出标志着 Google 首次将训练与推理产品线分离。基于同一 TensorCore 微架构，Google 同时推出了双核的 TPU v4（面向大规模训练）和单核的 TPU v4i（面向数据中心推理）[3]。这一" 训推分家" 策略使推理芯片摆脱了训练芯片高功耗、液冷的部署约束，为全球分布式推理服务提供了更高的性价比。Jouppi 等人在 ISCA 2021 中系统总结了从三代 TPU 部署中提炼出的十条设计教训，这些经验直接塑造了 v4i 的架构决策。

1. 技术不均衡进步与 CMEM 的引入。从 45nm 到 7nm，逻辑电路能效提升了约 2.6 倍，但 SRAM 密度提升缓慢（仅 1.3–2.4 倍），DRAM 则依靠 HBM 封装技术实现了 6.3 倍的能效跃升。这种不均衡意味着" 逻辑相对免费"，而片上存储成为更珍贵的资源。v4i 充分利用 7nm 的逻辑密度，在单核内集成了 **4 个 128×128 的 MXU**（与 v3 双核各 2 个数量相同，但集中在单核以简化编译），同时将片上 SRAM 扩展至 **128MB CMEM**（Common Memory），占芯片面积约 28%。CMEM 的读写带宽分别高达 2000 GB/s 和 1000 GB/s，可将生产应用的 HBM 访问过滤掉 50%–98%。在 BERT 等嵌入密集型负载中，CMEM 使 Gather 操作提速约 13–15 倍，整体模型性能提升 1.7–2.2 倍。

2. perf/TCO 优先于 perf/CapEx。Google 发现，数据中心的总拥有成本（TCO）与系统热设计功耗（TDP）高度相关（$R = 0.99$）。因此 v4i 没有盲目追求峰值算力，而是将芯片 TDP 控制在 **175W**、系统 TDP 控制在 275W，并采用空气冷却。这与 v3 的 450W 液冷形成鲜明对比，也使 v4i 能够以标准服务器形态部署在全球任何数据中心。相比 TPU v3，v4i 的生产应用性能与其相当（约 1.9 倍于 v2），但 perf/TDP 提升了 **2.3 倍**，其中 CMEM 贡献约 1.5 倍，7nm 工艺贡献约 1.3 倍。

3. 向后 ML 兼容性与推理浮点需求。 v4i 保留了 bfloat16 和 IEEE fp32，同时增加了 int8 支持。这一决策源于教训：强制量化会延迟模型部署（某些应用需要数月才能恢复因 int8 带来的质量损失），而向后兼容性确保了在 TPU v3 上训练的模型可以无缝部署到 v4i。在 MLPerf Inference 0.5–0.7 基准测试中，v4i 全程使用 bfloat16，相比 NVIDIA T4（ResNet/SSD 用 int8、NMT 用 fp16）平均速度为 **1.3–1.6 倍**，perf/TDP 约为 0.9–1.0 倍（NMT 场景下为 1.3 倍）。

4. 多租户、P99 延迟与 DNN 增长。 Google 超过 80% 的生产推理负载需要多租户运行（同时加载数十个模型），且推理的服务等级目标（SLO）由 **P99 延迟**决定，而非 batch size。大容量 CMEM 使编译器可以通过预取将权重暂存于片上，从而将多租户上下文切换时间控制在约 150 微秒。此外，Google 内部生产 DNN 的内存和计算需求自 2016 年以来以每年约 **1.5 倍**的速度增长，这要求 DSA 必须为未来预留 headroom。v4i 的 128MB CMEM 和 4D Tensor DMA 正是为了应对这一持续增长而设计的。

从架构演进的角度看，TPU v4i 是 Ironwood（v7）之前最早的" 推理回归" 预演。它首次证明了：在 DSA 世代内通过" 同一核心、两种封装" 实现训推分家，可以在不牺牲向后兼容性的前提下，针对推理场景的风冷、低 TDP、低尾延迟需求进行深度优化。

4.3 光路交换（OCS）技术原理

光路交换（OCS）是 TPU v4 架构革命的核心技术。与电交换不同，OCS 通过物理切换光信号的路径来实现互连拓扑的重构，而非将光信号转换为电信号进行处理。这一根本性的差异带来了带宽、功耗和延迟的多重优势。

在 TPU v4 的 OCS 实现中，芯片发出的光信号通过光纤传输到 OCS 交换机。OCS 交换机内部采用微机电系统（MEMS）技术，通过微小的镜面阵列将输入光信号反射到任意输出端口。这种全光交换机制的关键优势在于其比特率透明性：无论信号的数据速率是多少，OCS 都可以在不解析信号内容的情况下完成交换。这意味着 OCS 的带宽能力可以随着光模块技术的进步而自然提升，无需更换交换硬件。

TPU v4 的 OCS 系统支持微秒级的拓扑重构时间。这一快速的切换能力使得系统可以根据工作负载需求动态调整互连拓扑。例如，在进行数据并行训练时，系统可以配置为全连接的 All-Reduce 优化拓扑；在进行流水线并行训练时，可以配置为相邻阶段之间高带宽连接的线性拓扑。这种灵活性是传统电互连无法实现的。

从成本和功耗角度，OCS 同样展现出显著优势。根据 Jouppi 等人在 ISCA 2023 论文中的报告，TPU v4 系统中的 OCS 和相关光组件仅占系统总成本的不到 5%，功耗占比不到 3%。相比之下，同等规模的 InfiniBand 互连系统的成本和功耗占比通常要高得多。这一经济性优势源于光组件的共享复用：OCS 交换机可以被多个 TPU Pod 共享，根据需求动态分配带宽资源。

4.4 动态拓扑与 3D 环面

TPU v4 支持多种互连拓扑配置，包括传统的 2D 环面、扭曲 3D 环面（Twisted 3D Torus）以及自定义拓扑。这些拓扑选项通过 OCS 的动态重构能力实现，为不同的并行策略提供了最优的通信支持。

2D 环面拓扑是 TPU v2/v3 使用的经典配置，每个芯片与四个相邻芯片相连，形成一个二维网格，边缘连接形成环面。这种拓扑在数据并行训练中表现良好，All-Reduce 操作可以通过环面中的多条路径并行进行，充分利用网络带宽。

扭曲 3D 环面是 TPU v4 引入的新拓扑，通过在第三维度增加连接，将 2D 环面扩展为 3D 结构。这种拓扑减少了多跳通信的平均跳数，降低了通信延迟。对于大规模 All-Reduce 操作，扭曲 3D 环面可以提供更高的带宽利用率。

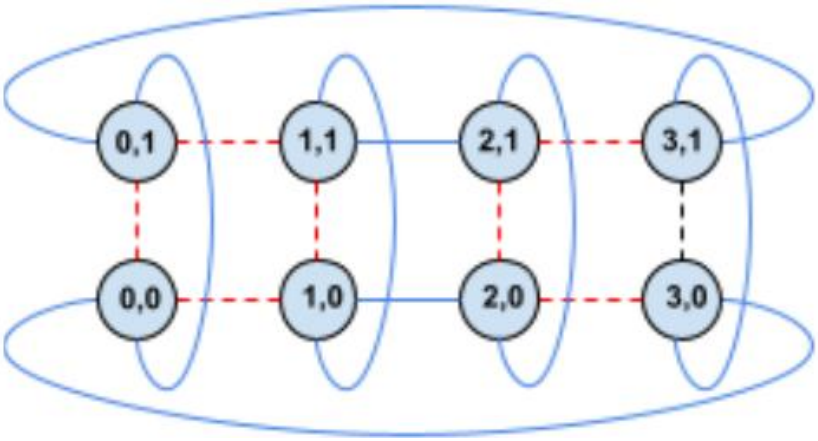

图 6: TPU v4 扭曲 3D 环面（Twisted Torus）拓扑示意图。通过在第三维度引入错位连接，减少了芯片间通信的平均跳数，提升了大规模集群的 All-Reduce 效率。来源：Jouppi et al., ISCA 2023

Chips	<64		64	
Regular Tori	1×1×1 (1)	2.1%	4×4×4 (64)	13.9%
	1×1×2 (2)	0.4%		
	1×2×2 (4)	6.7%		
	2×2×2 (8)	4.7%		
	2×2×4 (16)	6.4%		
	2×4×4 (32)	8.9%		
Total %		29%		14%
Chips	128-192		256-384	
Twisted Tori	4×4×8_T (128)	16.0%	4×8×8_T (256)	9.2%
Twistable, not twisted Tori	4×4×8_NT (128)	1.5%	4×8×8_NT (256)	1.5%
Regular Tori	4×4×12n (192)	0.7%	4×4×16 (256)	1.0%
			4×8×12 (384)	0.1%
Total %		18%		12%
Chips	512-768		1024-1536	
Twisted Tori			8×8×16_T (1K)	1.8%
Twistable, not twisted Tori			8×8×16_NT (1K)	1.4%
Regular Tori			4×16×16 (1K)	0.3%
	8×8×8 (512)	9.6%	4×4×64 (1K)	0.1%
	4×8×16 (512)	1.7%	4×8×32 (1K)	0.1%
	4×4×32 (512)	0.6%	8×12×16 (1.5K)	0.1%
	8×8×12 (768)	0.7%	4×4×96 (1.5K)	0.1%
			8×8×24 (1.5K)	0.1%
Total %		13%		4%
Chips	2048-3072			
Twisted Tori	8×16×16_T (2K)			1.4%
Twistable, not twisted Tori	8×16×16_NT (2K)			0.3%
Regular Tori	12×16×16 (3K)			5.7%
	4×4×192 (3K)			0.4%
Total %				8%

图 7: TPU v4 支持的拓扑配置汇总表，包括 Regular Tori（常规环面）、Twisted Tori（扭曲环面）和 Twistable Tori（可扭曲环面）等不同规模的配置选项。来源：Jouppi et al., ISCA 2023

除了标准拓扑，TPU v4 还支持用户自定义拓扑。通过软件配置 OCS 交换机，研究人员可以实验各种非传统拓扑，如胖树（Fat Tree）、超立方体（Hypercube）或针对特定模型结构优化的自定义拓扑。这种灵活性为 AI 系统的研究创新提供了宝贵的实验平台。

4.5 SparseCore：嵌入式专用加速

TPU v4 的另一个重要创新是引入了 SparseCore，一个专门用于加速 Embedding 查找和稀疏计算的专用处理器。这一创新的背景是推荐系统和自然语言处理模型对大规模 Embedding 表的广泛使用。

在现代推荐系统中，用户和物品的 Embedding 表可能包含数十亿甚至数千亿条目，每个条目是数百维的向量。在训练和推理过程中，需要频繁地进行 Embedding 查找（根据 ID 检索对应的向量）和梯度更新（根据损失更新 Embedding 向量）。这些操作虽然计算量不大，但涉及大规模的随机内存访问，对传统的稠密计算引擎（如脉动阵列）效率极低。

SparseCore 针对这一工作负载进行了专门优化。每个 TPU v4 芯片集成了一个 SparseCore，包含专门优化的大容量 SRAM 和用于 Embedding 查找的专用硬件逻辑。如图 8所示，SparseCore 由序列调度器（Sequencer）、分发单元（Dispatch）和 16 个处理单元（Tile）组成，每个 Tile 包含稀疏 VPU（scVPU）、Spmem 存储体和访存控制逻辑。

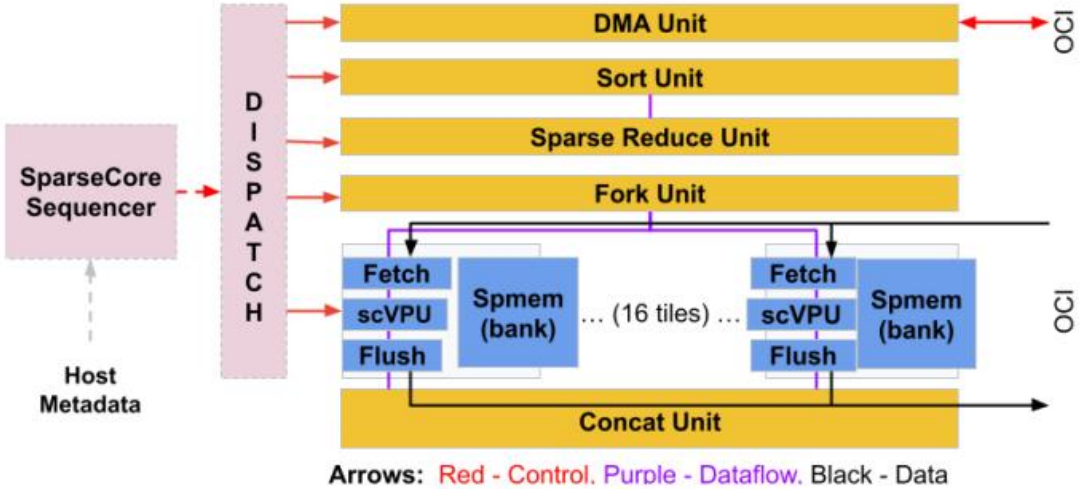

图 8: TPU v4 SparseCore 架构框图，展示了序列调度器、分发单元、16 个 Tile（含 scVPU 和 Spmem）以及 ICI 互连接口。来源：Jouppi et al., ISCA 2023

SparseCore 与主脉动阵列（DenseCore）协同工作：DenseCore 负责稠密矩阵运算（如 Transformer 的注意力计算），SparseCore 负责 Embedding 相关的稀疏操作。根据 Google 的测量数据，SparseCore 可以将 Embedding 密集型工作负载的性能提升 5 至 7 倍，而仅占芯片面积和功耗的约 5%。这种专用加速单元的引入，反映了 DSA 设计哲学的进一步延伸：不仅在芯片层面进行领域专用优化，还在芯片内部针对不同计算模式进行更细粒度的专用化。

4.6 4096 芯片超算系统

TPU v4 超算系统的最大规模达到 4096 个芯片，通过 64 个机架部署，每个机架包含 64 个 TPU v4 芯片。这一规模的实现得益于 OCS 技术的可扩展性和模块化设计。图 9展示了一个完整的 TPU v4 Pod 机架阵列。

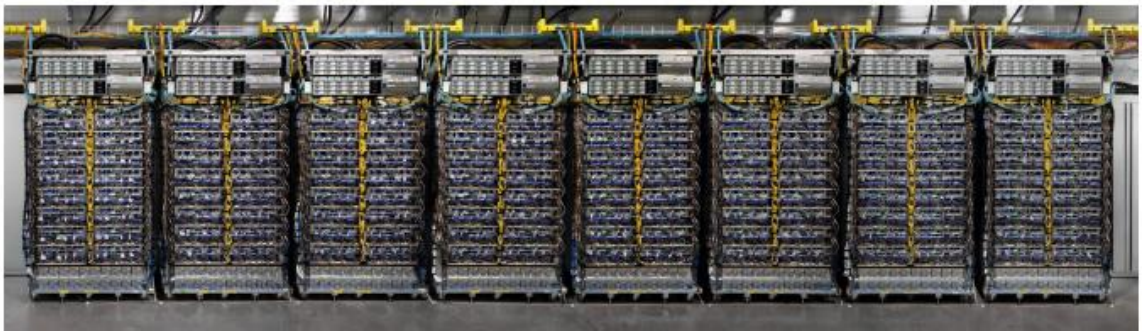

图 9: TPU v4 Pod 实物照片，展示 64 个机架组成的超算系统。来源：Jouppi et al., ISCA 2023

在物理部署层面，TPU v4 采用了液冷技术以支持高功率密度。每个 TPU v4 芯片的功耗约为 200 瓦，4096 芯片系统的总功耗达到约 800 千瓦，需要精密的散热设计。液冷系统不仅提供了比风冷更高的散热效率，还降低了数据中心的整体能耗，因为不再需要大量的空调制冷。

在软件层面，Google 开发了专用的集群管理和任务调度系统，支持大规模训练任务的自动并行化、容错处理和弹性扩展。当某个芯片或链路发生故障时，系统可以自动将任务迁移到备用资源，避免训练中断。这种高可用性设计对于需要数天甚至数周的大规模训练任务至关重要。

4.7 性能对比与能效分析

根据 ISCA 2023 论文中的系统性能评估，TPU v4 相比前代和竞品展现出显著的性能优势。如图 10所示，在不同系统规模下，TPU v4 相比 NVIDIA A100 和 Graphcore IPU 展现出良好的性能扩展性。

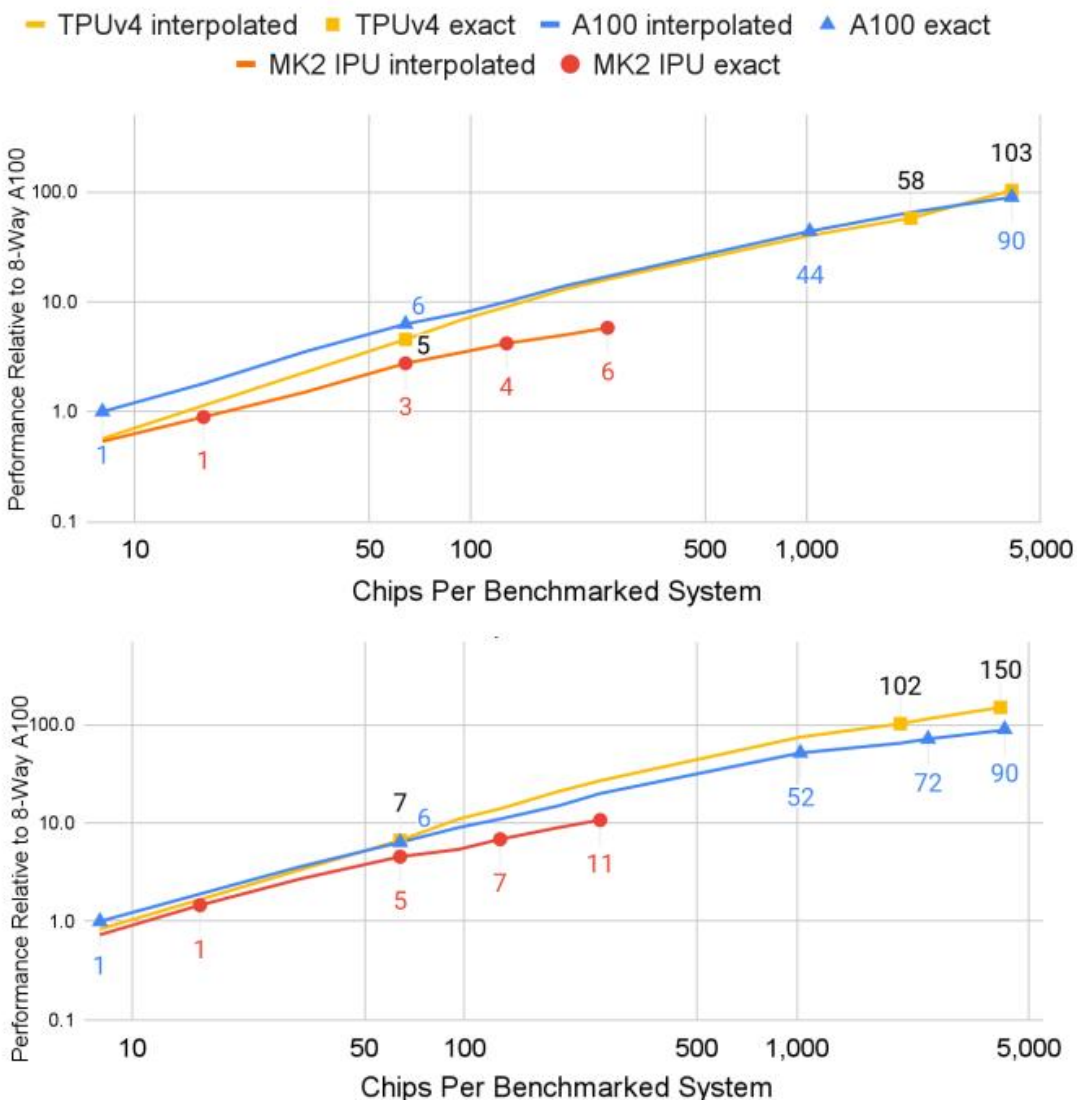

图 10: TPU v4、NVIDIA A100 和 Graphcore IPU 在不同芯片规模下的性能扩展对比。上图展示了相对 8 路 A100 的性能（对数坐标），下图展示了绝对性能指标。来源：Jouppi et al., ISCA 2023

与 TPU v3 相比，单芯片 TPU v4 的性能提升 2.1 倍，能效比提升 2.7 倍。这一提升源于工艺进步（从 16nm 到 7nm）、架构优化（更大的 MXU、更高的时钟频率）和内存升级（HBM2e）的综合作用。在系统级性能方面，4096 芯片的 TPU v4 超算系统相比 1024 芯片的 TPU v3 系统，整体训练吞吐量提升约 10 倍，接近线性扩展。这一高效率得益于 OCS 互连的高带宽和低延迟，以及 XLA 编译器对大规模并行的有效优化。

与竞品相比，TPU v4 在相似规模的系统配置下，相比 NVIDIA A100 展现出 1.2 至 1.7 倍的速度优势，同时功耗降低 1.3 至 1.9 倍。相比 Graphcore IPU Bow，TPU v4 的速度优势达到 4.3 至 4.5 倍。这些对比数据表明，TPU v4 的专用架构设计和系统级优化在大规模 AI 训练场景下具有显著优势。

在能效和可持续性方面，TPU v4 部署在 Google 优化设计的仓储级数据中心中，综合能源效率（PUE）约为 1.1，远低于传统数据中心的 1.5-2.0。根据 Google 的估算，在 TPU v4 上训练大型语言模型，相比在典型本地数据中心使用当代加速器，可以减少约

20 倍的碳排放。这一环境效益对于负责任地发展大规模 AI 技术具有重要意义。

5 TPU v6 (Trillium) 与 v7 (Ironwood)：推理时代的回归与放大

5.1 Trillium：训练与推理的双重飞跃

2024 年 12 月发布的 Trillium (TPU v6) 是 Google 宣称的 “迄今最强 AI 加速器”，被用于训练 Gemini 2.0 [9]。根据 Google 官方白皮书，Trillium 相比前代 v5e 的主要提升包括：

- 训练性能提升 $> 4\times$，推理吞吐提升最高 $3\times$；
- 能效提升 67%，单芯片峰值算力提升 $4.7\times$；
- HBM 容量翻倍，支持 host offloading；
- 第三代 SparseCore，嵌入密集型模型性能提升 $2\times$。

Trillium 在大规模扩展上表现突出：在 12 个 pod（3072 芯片）的部署中实现了 99% 的弱扩展效率，在 24 个 pod（6144 芯片）上预训练 GPT-3 175B 模型时仍保持 94% 的扩展效率。此外，Trillium 可借助 Jupiter 网络结构扩展至超过 10 万颗芯片。

然而，这些数据全部来自 Google 内部基准测试，缺乏独立第三方（如 MLPerf 提交中完整的可比配置）的验证。特别是在 “每美元训练性能提升 2.5 倍” 这一宣称中，价格因素与硬件性能被紧密耦合，难以剥离分析。

5.2 Ironwood：专为推理而生的第七代 TPU

2025 年 3 月发布的 Ironwood (TPU v7) 标志着 TPU 战略向推理的明确回归。Google 将其定位为首款 “purpose-built for inference” 的 TPU，面向 “会思考的 AI 模型”（如 Gemini 2.5 的 reasoning 任务） [8]。

Ironwood 的关键规格令人印象深刻：

- 单芯片 192 GB HBM（前代的 6 倍），带宽 7.37 TB/s；
- 片间互联 1.2 TBps；
- Pod 规模可达 9216 液冷芯片，整体 FP8 算力 42.5 Exaflops；
- 能效为 Trillium 的 2 倍。

从架构演进的角度看，Ironwood 代表了在 v4 超算互联能力基础上的 “推理放大”：巨大的单芯片内存容量使其能够容纳更大规模的模型或更长的上下文，而液冷和超高密度的 pod 配置则针对大规模在线推理服务进行了优化。这与 Anthropic 在 2025 年与 Google 签署的史上最大 TPU 采购协议——计划 2027 年扩展至 100 万颗 Trillium/Ironwood——形成了战略呼应 [7]。

5.3 规模部署与 Jupiter 网络

Trillium 和 Ironwood 的规模化离不开 Google 的 Jupiter 数据中心网络结构。Jupiter 提供 13 Petabits/s 的对分带宽，使得单一分布式训练作业可以跨越数十万颗加速器。这一 “超算即服务” 的模式，与 NVIDIA 的 DGX SuperPOD 形成了直接竞争。

6 软件栈、专利布局与横向对比

6.1 XLA 编译器架构

XLA（Accelerated Linear Algebra）是 TPU 软件生态系统的核心，承担着将高级机器学习框架的计算图转换为高效 TPU 机器码的关键任务。理解 XLA 的设计原理和优化策略，是把握 TPU 系统整体性能特征的重要环节。

XLA 的编译流程遵循经典的编译器架构，分为前端、中间表示（IR）优化和后端代码生成三个阶段。前端接收来自 TensorFlow、JAX 等框架的计算图，将其转换为 XLA 的高级中间表示（High-Level Operations, HLO）。HLO 是一种与硬件无关的函数式中间表示，支持张量运算、控制流和并行原语。

在 HLO 层面，XLA 执行一系列与目标硬件无关的优化变换。这些优化包括常量传播、公共子表达式消除、死代码消除、算子融合等经典的编译器优化技术。算子融合是其中最为关键的优化之一：通过将多个连续操作（如卷积、批归一化、激活函数）融合为单个融合内核，XLA 可以显著减少中间结果的内存访问，提高计算密度。

后端阶段是 XLA 与 TPU 硬件深度协同的关键。针对 TPU 的脉动阵列架构，XLA 需要将 HLO 中的矩阵运算映射为高效的脉动阵列执行计划。这涉及复杂的循环变换、数据分块（tiling）和内存布局优化。对于分布式训练场景，XLA 还需要生成高效的芯片间通信代码，将计算与通信重叠以隐藏延迟。

如图 11所示，现代 TPU 芯片（以 v4 为例）包含两个 TensorCore，每个 TensorCore 内有四个 MXU（脉动阵列）、VPU、VMEM、Scalar Unit 和 SMEM。XLA 编译器需要精确地将矩阵乘法映射到 MXU，将逐元素操作映射到 VPU，并安排 DMA 在 HBM 与片上内存之间的数据搬运。

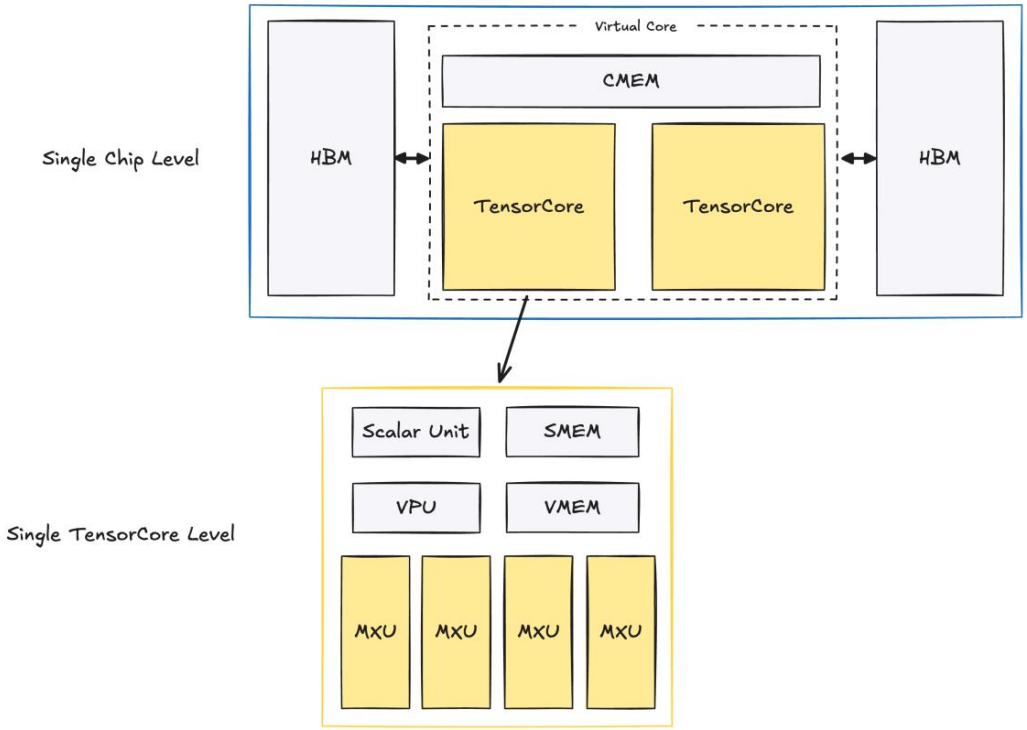

图 11: TPU v4 单芯片与单 TensorCore 架构框图。芯片级包含 HBM 和两个共享 CMEM 的 TensorCore；TensorCore 级包含 MXU、VPU、Scalar Unit 和各级片上内存。来源：June2, 2025

6.2 脉动阵列代码生成

将高层矩阵运算高效映射到 TPU 的脉动阵列，是 XLA 编译器最具挑战性的任务之一。这一过程涉及多个层次的优化决策。

在算法层面，XLA 需要将大矩阵乘法分解为适合脉动阵列尺寸的子块。TPU v4 的 MXU 是 128×128 的阵列，因此矩阵乘法需要被分解为 128×128 的子矩阵运算。分解策略需要考虑多个因素：子块的计算效率、数据复用机会、以及在不同子块之间切换的开销。XLA 采用启发式算法和成本模型来选择最优的分解策略。

在数据布局层面，XLA 需要优化张量在内存中的存储格式，以最大化脉动阵列的数据供给效率。对于矩阵乘法，这意味着确保权重矩阵在内存中按行或按列连续存储，以便脉动阵列可以高效地预加载和复用。XLA 还可以执行数据重排变换，将计算图改写为更适合硬件执行的形式。

在调度层面，XLA 需要精心安排计算的时序，以实现计算与数据加载的重叠。脉动阵列一旦启动，需要持续的数据流来维持高利用率。XLA 通过软件流水线技术，将数据加载、计算和数据写回三个阶段重叠执行，最大化硬件利用率。

6.3 分布式并行优化

对于大规模分布式训练，XLA 的优化策略更为复杂。XLA 需要自动或半自动地将计算图划分到数千个 TPU 芯片上，同时最小化芯片间通信开销。

数据并行是最常用的并行策略，适用于模型可以放入单芯片内存的情况。在数据并

行模式下，每个 TPU 芯片持有完整的模型副本，处理不同的数据批次。每轮迭代后，各芯片通过 All-Reduce 操作同步梯度。XLA 针对 TPU 的 ICI 互连优化 All-Reduce 实现，采用环型 All-Reduce 或树型 All-Reduce 算法，最大化带宽利用率。

模型并行适用于模型过大无法放入单芯片内存的情况。在模型并行模式下，网络的不同层被分配到不同芯片。XLA 需要分析计算图的依赖关系，将层间的激活值和梯度传递映射为高效的芯片间通信。对于 Transformer 等具有规则结构的模型，XLA 可以自动推断最优的层划分策略。

流水线并行将网络划分为多个阶段，每个阶段由一个或多个芯片负责。不同数据批次可以同时在不同阶段进行处理，形成流水线。XLA 需要平衡各阶段的计算负载，最小化流水线气泡（bubble）造成的效率损失。对于大规模语言模型，XLA 通常采用混合并行策略，结合数据并行、模型并行和流水线并行，以充分利用大规模 TPU Pod 的计算能力。

6.4 JAX 与函数式编程模型

JAX 是 Google 开发的机器学习研究框架，其设计哲学与 TPU 硬件特性高度契合。理解 JAX 的设计原理，有助于把握 TPU 软件生态的发展方向。

JAX 基于纯函数式编程模型，将神经网络计算表示为纯函数的变换组合。这与 TensorFlow 的声明式计算图模型形成了对比。函数式编程的优势在于其透明性和可组合性：纯函数没有副作用，编译器可以更安全地进行优化变换；函数组合提供了强大的抽象能力，支持复杂的程序变换。

JAX 提供了四个核心的程序变换原语：grad 用于自动微分，jit 用于即时编译优化，vmap 用于自动向量化，pmap 用于自动并行化。这些变换可以任意组合，构建出复杂的优化和并行策略。例如，jit(grad(model_loss)) 表示对模型损失函数进行自动微分并即时编译；pmap(vmap(model_forward)) 表示在多个设备上并行执行模型的向量化前向传播。

在 TPU 上，JAX 的计算被 XLA 编译为高效的机器码。JAX 的函数式风格便于 XLA 进行激进的优化，因为函数之间的数据依赖关系更加显式。JAX 的并行原语（pjit、pmap）直接映射到 TPU 的 Pod 拓扑，开发者可以用高层次的抽象表达复杂的分布式并行逻辑，而无需手动处理底层的通信和同步细节。

6.5 Pathways 运行时与超大规模分布式执行

Pathways 是 Google 为超大规模分布式计算设计的运行时系统，它支持跨数十万颗芯片的单一计算图执行。Ironwood 的发布进一步强调了 Pathways 与 TPU 硬件的深度集成。Pathways 的核心理念是将计算资源抽象为一个统一的虚拟加速器池，使得开发者可以用单设备编程的思维方式编写在超大规模集群上运行的程序。运行时负责自动的并行化、通信调度和容错恢复，将底层分布式系统的复杂性完全隐藏。

6.6 软件生态的比较分析

将 TPU 的软件栈与 NVIDIA 的 CUDA 生态进行对比，可以揭示 DSA 与通用 GPU 在软件策略上的本质差异。

CUDA 生态的核心是底层编程能力。CUDA C/C++ 允许开发者直接控制 GPU 的线程层次结构、内存层次结构和同步原语，编写高度优化的内核代码。这种底层控制能力带来了极高的性能上限，但也要求开发者具备深厚的架构知识。CUDA 生态还包括丰富的库（cuDNN、cuBLAS、NCCL 等）和工具（Nsight Profiler 等），为性能优化提供了强大支持。

TPU 的软件栈则走了一条不同的道路。XLA 和 JAX 提供了更高层次的抽象，将性能优化的责任主要交给编译器。这种策略的优势在于开发效率和可移植性：开发者可以用简洁的 Python 代码表达模型逻辑，编译器自动生成高效的 TPU 代码；同一份代码可以在单芯片、多芯片或大规模 Pod 上运行，无需手动调整并行策略。

然而，这种高层抽象也带来了一定的局限性。对于某些非常规的计算模式，编译器可能无法生成最优代码，而开发者又缺乏直接控制硬件的手段。此外，TPU 生态的封闭性——只能在 Google Cloud 上使用，主要支持 TensorFlow 和 JAX——限制了其用户基础和创新速度。

这种软件策略的差异反映了两种不同的设计理念：CUDA 追求极致的性能和灵活性，即使需要更高的开发门槛；TPU 则追求开发效率和系统级优化，即使牺牲一定的灵活性。随着 AI 模型和工作负载的多样化，这两种策略各有其适用场景，未来的软件生态可能呈现多元共存的格局。

6.7 专利布局：开放论文与收敛专利的反差

与大量公开的顶会论文形成鲜明对比的是，Google 在核心 TPU 硬件架构上的美国专利布局相对稀疏，呈现出 “论文开放、专利收敛” 的策略。根据最新的专利分析，六项高相关度专利的分布如下：

- **脉动阵列旋转数据**（US20160342893A1，Google）：Ross 等人提出的卷积核旋转机制，将 CNN 计算映射到脉动阵列；
- **神经架构搜索控制器**（US20230368024A1，Google）：Zoph 等人的 NAS 控制器专利；
- **分层分解搜索空间的 NAS**（US12293276B2，Google）：Tan 等人的 MobileNet 搜索空间专利；
- **MEMS 光电路交换开关**（US11506884B2，Molex）：Hu 等人的 MEMS OCS 专利，是 TPU v4 光互连的物理基础；
- **BFLOAT16 算术指令**（US20230069000A1，Intel）：Intel 主导的 bfloat16 指令集专利；
- **可重构脉动阵列编译器优化**（US20250028786A1，SambaNova）：SambaNova 在数据流 stationary 模式选择上的编译器专利。

这一专利分布揭示了两个重要事实：首先，Google 直接持有的核心硬件专利仅覆盖

脉动阵列数据旋转和 NAS 算法，而 bfloat16 和 OCS 等关键技术分别由 Intel 和 Molex 掌控；其次，编译器层面的数据流调度专利开始被 SambaNova 等新兴厂商补充，预示着 DSA 竞争的战场正在从硬件向软件-编译器栈转移。

6.8 与 NVIDIA GPU 的架构对比

TPU 与 NVIDIA GPU 代表了 AI 加速器的两种截然不同的设计哲学。深入理解这两种架构的差异，有助于把握 DSA 与通用 GPU 在 AI 计算领域的竞争格局和发展趋势。

在执行模型层面，TPU 采用数据流驱动的脉动阵列执行，而 GPU 采用 SIMT（单指令多线程）模型。TPU 的脉动阵列通过规则的数据流动实现极高的计算密度和能效，但牺牲了通用性和灵活性。GPU 的 SIMT 模型支持复杂的控制流和细粒度的线程调度，可以高效处理多样化的并行任务，但控制开销和内存访问开销较高。

表 1: TPU v6e 与 NVIDIA H100 的关键规格对比（基于公开资料）

指标	**TPU v6e (8-chip)**	**NVIDIA H100 NVL (4-GPU)**
峰值算力	7,344 TFLOPS	6,682 TFLOPS
HBM 容量	256 GB	376 GB
TDP (估算)	~300W/芯片	700W/GPU
扩展规模	4,096+ 芯片 (pod)	256 GPU (SuperPOD)
编程模型	JAX/TensorFlow/XLA	CUDA/PyTorch/TensorFlow
云上可用性	GCP 独占	多云/本地

在内存系统层面，TPU 采用简化的内存层次结构，主要由 HBM 和软件管理的片上 SRAM 组成，去除了复杂的硬件缓存。这种设计降低了内存访问的不确定性，但需要编译器进行精细的内存管理。GPU 则具有复杂的缓存层次结构（L1、L2）和程序员可见的共享内存，提供了更多的优化灵活性，但也增加了编程复杂性。

在互连架构层面，TPU v4 采用专用的光互连技术，支持动态拓扑重构，针对 AI 训练的通信模式进行了深度优化。NVIDIA GPU 采用 NVLink 和 NVSwitch 技术，提供高带宽的点对点连接，但拓扑相对固定。两种技术各有优势：TPU 的光互连在超大规模集群上具有成本和功耗优势，NVLink 在中小规模集群上具有成熟度和生态优势。

在软件生态层面，CUDA 提供了底层的硬件控制能力，支持高度优化的内核开发，但需要更多的开发投入。TPU 的软件栈以编译器优化为核心，提供了更高的开发效率，但牺牲了一定的控制灵活性。两种生态的成熟度也存在差异：CUDA 经过 15 年以上的发展，拥有庞大的开发者社区和丰富的库支持；TPU 生态相对年轻，主要在 Google Cloud 上提供服务。

6.9 MLPerf 基准测试分析

MLPerf 是业界广泛认可的机器学习性能基准测试套件，提供了客观比较不同 AI 加速器性能的标准。分析 TPU 在 MLPerf 测试中的表现，可以验证其架构设计在实际工

作负载上的效果。

在 MLPerf Training v2.0 和 v3.0 的测试结果中，TPU v4 在多个基准测试上展现出强劲性能。在 BERT-Large（用于自然语言理解的 Transformer 模型）训练任务上，4096 芯片的 TPU v4 系统达到了约 1.5 分钟的训练时间，与同等规模的 NVIDIA A100 系统相当或略有优势。在 ResNet-50（用于图像分类的卷积网络）训练任务上，TPU v4 同样展现出与 A100 相当或更好的性能。

值得注意的是，TPU 在大型语言模型训练上的优势更为明显。在 GPT-3 级别（1750 亿参数）的模型训练上，TPU v4 的专用架构和光互连系统展现出更高的扩展效率和更低的通信开销。这反映了 TPU 架构设计对 Transformer 类模型的针对性优化。

在能效方面，TPU v4 在 MLPerf 测试中通常展现出比 GPU 更好的性能每瓦特指标。这一优势源于 TPU 的专用架构设计：脉动阵列的高计算密度、光互连的低功耗特性、以及系统级的能效优化。对于大规模数据中心部署而言，能效优势可以转化为显著的运营成本节约和环境效益。

6.10 TCO 与部署成本分析

总拥有成本（Total Cost of Ownership, TCO）是企业选择 AI 基础设施时的关键考量因素。根据 Google 和第三方分析机构的估算，TPU 在特定场景下可以显著降低 AI 训练和推理的 TCO。

TPU 的 TCO 优势主要来自几个方面。首先是计算效率：TPU 的专用架构在执行神经网络工作负载时具有更高的效率，意味着完成同样的任务需要更少的芯片时间和能源消耗。其次是系统级优化：TPU v4 的光互连系统在超大规模部署上具有成本和功耗优势，OCS 交换机的共享复用降低了每个芯片的网络成本。第三是运营效率：TPU 作为云服务提供，用户无需承担硬件采购、部署和维护的固定成本，可以根据需求弹性扩展。

然而，TPU 的 TCO 优势并非适用于所有场景。对于小规模部署或通用计算需求，GPU 的成熟生态和灵活编程模型可能带来更高的开发效率和更低的迁移成本。此外，TPU 的云绑定模式虽然降低了固定成本，但也意味着长期使用的累计费用可能超过自建基础设施。

根据 Semianalysis 等第三方机构的分析，在大规模 AI 训练场景下，TPU v4 的 TCO 相比同等规模的 NVIDIA GPU 系统可以降低约 30-50%。这一优势在大型科技公司的大规模训练任务中尤为明显，但对于中小规模用户或通用计算需求，GPU 可能仍然是更经济的选择。

值得注意的是，2025 年已有多个实际迁移案例支持 TPU 的经济性：Midjourney 将图像生成推理从 GPU 迁移至 TPU v6e 后，月度支出从 210 万美元降至 70 万美元（降幅 65%）；Anthropic 签署了 Google 史上最大 TPU 采购协议，计划 2027 年扩展至 100 万颗 Trillium/Ironwood。然而，这些案例大多来自与 Google 有战略合作的客户，独立第三方的大规模基准对比仍然稀缺。

6.11 性能瓶颈与优化策略

尽管 TPU 在设计上进行了深度优化，但在实际应用中仍然可能遇到性能瓶颈。识别和解决这些瓶颈，是发挥 TPU 系统潜力的关键。

内存带宽瓶颈是最常见的问题之一。虽然 TPU 配备了高带宽的 HBM，但在某些工作负载下，计算速度仍然可能超过数据供给速度。这种情况下，性能受限于内存带宽而非计算能力。解决策略包括：增加批处理大小以提高数据复用、优化数据布局以改善局部性、以及使用 XLA 的算子融合减少中间结果的内存访问。

通信瓶颈在大规模分布式训练中尤为突出。虽然 TPU 的光互连提供了高带宽，但 All-Reduce 等集合通信操作在数千芯片规模下仍可能成为瓶颈。优化策略包括：采用混合并行策略减少全局同步的频率、使用梯度压缩技术减少通信量、以及通过 XLA 的通信-计算重叠隐藏延迟。

负载不均衡是流水线并行中的常见问题。如果网络的某些层计算量远大于其他层，会导致流水线气泡增加，降低整体效率。解决策略包括：精心设计流水线划分以平衡各阶段的计算量、使用微批次（micro-batching）提高流水线利用率、以及采用动态调度算法适应变化的负载。

利用率不足是另一个常见问题。对于小规模模型或低批处理场景，TPU 的强大计算能力可能无法充分利用。这种情况下，考虑使用较小的 TPU 配置（如 TPU v4i）或增加批处理大小，可能获得更好的成本效益 [3]。

7 批判性讨论：成就、代价与未解张力

7.1 资料不对称与方法论的局限

撰写 TPU 综述面临的最大挑战是学术验证的 “资料不对称”。Jouppi 2017、Norrie 2021 和 Jouppi 2023 三篇论文经过了严格的同行评审，是可靠的学术来源。然而，对于 2024 年后的 Trillium 和 Ironwood，目前仅有 Google 官方博客和白皮书，缺乏独立的第三方基准测试、详细的微架构披露和完整的功耗数据。

这意味着我们对 TPU 最新代际的讨论，不可避免地带有 “基于宣称” 的色彩。作为学术综述，我们必须在每个结论前标注其证据强度：是经过同行评审的实测数据，还是厂商的市场宣称？本文已尽量区分这两类来源，但读者在引用最新数据时应保持审慎。

7.2 封闭生态的长短期权衡

TPU 的封闭性是另一把双刃剑。一方面，Google 通过控制从芯片设计、制造、数据中心、网络结构到编译器、框架、模型的全栈，实现了极致的垂直优化。这种 “端到端控制” 是 TPU 能够在特定负载上超越通用 GPU 的关键原因。另一方面，TPU 仅在 Google Cloud 上可用（on-premise 销售仍处于早期阶段），这意味着：

- 用户面临严重的供应商锁定风险；
- 研究者难以在 Google 生态系统之外独立复现 TPU 上的结果；

- 框架支持上，JAX 虽与 TPU 深度整合，但 PyTorch/XLA 的成熟度仍落后于原生 CUDA。

7.3 脉动阵列与稀疏性、动态性的张力

TPU 的脉动阵列架构在稠密矩阵乘法上表现出色，但对稀疏矩阵和不规则数据流的支持天然受限。随着混合专家模型 (MoE) 和动态推理（如 chain-of-thought reasoning）的兴起，如何在脉动阵列上高效支持稀疏性和动态形状，成为 TPU 面临的长期架构挑战。Ironwood 虽然通过巨大的单芯片内存缓解了部分问题，但并未从根本上改变脉动阵列的计算范式。

7.4 能效宣称的归因问题

Jouppi 2023 中 20× 碳排放降低的宣称 [6]，混合了加速器能效、数据中心 PUE 和电力结构三个因素。将这一成就完全归因于 TPU 架构是不严谨的。类似地，Ironwood“能效为 Trillium 2 倍” 的宣称，也未公开具体的测试负载、测量方法和环境条件。在缺乏这些细节的情况下，跨代际、跨厂商的能效比较应被视为初步估算而非定论。

8 未来趋势与开放问题

8.1 从训练到推理的范式转变

TPU v7（代号 Ironwood）的发布标志着 Google 在 AI 加速器设计策略上的重大转变：从训推兼顾重新聚焦于推理专用优化。这一转变反映了 AI 计算市场格局的深刻变化。

大语言模型训练的成本已经上升到只有少数科技巨头能够承担的水平。训练 GPT-4 级别的模型可能需要数亿美元的计算资源，这种规模的投入限制了训练市场的参与者数量。相比之下，推理市场的需求正在爆发式增长：随着 ChatGPT 等应用的普及，全球每天的 AI 推理请求量已达到数千亿级别。

推理工作负载与训练有本质的不同。训练通常采用大批量（large batch）以最大化计算效率，而推理通常需要处理单个请求或微小批量（small batch），对延迟极为敏感。训练可以容忍秒级甚至分钟级的响应时间，而推理需要毫秒级的延迟以保证用户体验。这些差异导致了不同的优化目标：训练追求高吞吐量和能效，推理追求低延迟和高性价比。

TPU v7 针对这些推理特性进行了专门优化。根据 Google 在 2025 年发布的资料，Ironwood 相比前代在推理场景下实现了显著的性能提升和成本降低。这一专用化趋势预示着 AI 加速器市场可能进一步细分：训练市场由少数通用加速器主导，推理市场则出现更多针对特定场景的专用解决方案。

8.2 稀疏性与低精度计算

神经网络模型中普遍存在的稀疏性——即大量权重或激活值为零——为架构优化提供了重要机会。未来的 TPU 架构可能会加强对稀疏计算的硬件支持。

结构化稀疏性是目前硬件支持最成熟的稀疏形式。NVIDIA Ampere 架构引入的 2:4 结构化稀疏性模式（每 4 个元素中有 2 个为零）已经被证明可以在保持模型质量的同时实现 2 倍的加速。TPU 未来的架构可能会引入类似甚至更灵活的结构化稀疏支持。

非结构化稀疏性的硬件支持更为复杂，因为需要处理不规则的内存访问模式。然而，随着剪枝技术的进步，非结构化稀疏模型的比例正在增加。未来的加速器可能会引入专门的稀疏计算单元，支持高效的稀疏矩阵乘法和稀疏向量运算。

低精度计算是另一个重要的优化方向。从 FP32 到 FP16/BF16，再到 INT8 和 FP8，神经网络对数值精度的需求正在持续降低。TPU v4 已经支持了 FP8 格式，未来的架构可能会进一步降低到 INT4 甚至更低的精度。然而，低精度的推广需要解决数值稳定性和模型质量保持的挑战，这涉及硬件设计、算法创新和训练技术的协同进步。

8.3 存算一体与近内存计算

随着 AI 模型规模的持续增长，内存墙（Memory Wall）问题日益突出。存算一体（Processing-in-Memory, PIM）和近内存计算（Near-Memory Computing）技术为突破这一瓶颈提供了可能路径。

存算一体的核心思想是将计算单元直接集成到内存芯片中，在数据存储位置附近执行计算，避免数据的长距离搬运。对于 Embedding 查找等内存密集型操作，PIM 可以显著降低延迟和功耗。三星、SK 海力士等内存厂商已经推出了基于 HBM-PIM 的原型产品，虽然这些技术的大规模应用仍需时日，但代表了未来的发展方向。

近内存计算是一种相对保守但更容易实现的方案，通过在内存控制器附近增加计算逻辑，减少数据搬运距离。TPU 的 SparseCore 可以视为一种近内存计算的形式：它将 Embedding 表存储在靠近计算单元的专用 SRAM 中，加速随机查找操作。

对于 TPU 架构而言，进一步整合 PIM 技术可能带来显著的性能提升。例如，将 Embedding 查找操作完全下推到 HBM 内部的计算单元，可以释放主计算引擎专注于稠密矩阵运算。这种异构计算架构需要硬件、编译器和算法的协同设计，是一个富有前景的研究方向。

8.4 软件生态的开放与标准化

TPU 生态系统的封闭性是其广泛采用的主要障碍之一。展望未来，软件生态的开放与标准化可能是 TPU 发展的关键议题。

目前，TPU 只能在 Google Cloud Platform 上使用，且主要支持 TensorFlow 和 JAX 框架。这种封闭模式虽然有利于 Google 控制用户体验和优化端到端性能，但也限制了用户选择和第三方创新。PyTorch 等其他主流框架对 TPU 的支持相对有限，增加了用户的迁移成本。

一种可能的发展方向是逐步开放 TPU 的软件栈，允许第三方开发者和研究机构为 TPU 开发优化库和工具。另一种可能是推动 AI 加速器接口的标准化，类似于 GPU 领域的 OpenCL 或 Vulkan，允许统一的编程接口适配不同的硬件后端。

无论采取何种路径，软件生态的健康发展对于 TPU 的长期成功至关重要。硬件性能的优势需要通过易用的软件接口才能转化为实际的用户价值。如何在保持差异化竞争优势的同时促进生态繁荣，是 Google 在 TPU 发展中需要平衡的关键问题。

8.5 可持续性与绿色计算

随着 AI 模型规模和训练能耗的持续增长，AI 计算的可持续性已成为不可忽视的议题。TPU 在设计中已经开始考虑环境可持续性，这一趋势在未来可能会进一步加强。

TPU v4 的能效优势直接转化为更低的碳排放。根据 Google 的估算，在优化设计的数据中心中使用 TPU v4 训练大型模型，相比在典型本地数据中心使用当代加速器，可以减少约 20 倍的碳排放。这一优势源于多个因素：TPU 的高能效设计、Google 数据中心的可再生能源使用、以及系统级的能效优化。

未来的 TPU 架构可能会进一步加强对可持续性的考量。这包括：更激进的低功耗设计、对可再生能源供电波动的适应能力、以及更长的硬件生命周期以减少电子垃圾。此外，Google 可能会开发专门的工具和方法论，帮助用户评估和优化其 AI 工作负载的碳足迹。

可持续性的考量不仅影响硬件设计，也可能影响算法和软件的发展方向。例如，推动更高效的模型架构（如稀疏模型、知识蒸馏模型）、开发碳感知的训练调度算法、以及建立 AI 计算的环境影响评估标准。这些努力需要硬件厂商、云服务提供商、研究机构和政策制定者的协同合作。

9 结论

Google TPU 从 2015 年部署至今的十年演进，为领域专用架构（DSA）的设计提供了宝贵的工程经验与理论启示。本文系统梳理了 TPU 各代的架构演进：

TPU v1 通过极简主义的脉动阵列设计，在数据中心推理场景中实现了相对于同期 CPU/GPU 数量级的性能与能效优势，确立了“用确定性执行替代动态调度”的 DSA 设计范式。

TPU v2/v3 在“快速上市”的工程约束下，成功将 TPU 从推理加速器扩展为训练平台。bfloat16、HBM、ICI 互联和可编程 VPU 的引入，为后续的大规模模型训练奠定了基础。

TPU v4 代表了 AI 加速器架构的重大飞跃。光路交换（OCS）技术使 4096 芯片的超算互联拓扑可以动态重构，SparseCore 以极低的面积代价解决了嵌入密集型负载的瓶颈。同一世代推出的 TPU v4i 则首次实践了“训推分家”策略，通过大容量 CMEM、空气冷却和 P99 延迟优化，为数据中心推理树立了新的能效标杆。

TPU v6 (Trillium) 与 v7 (Ironwood) 将规模与推理性能推向新高度，但在学术

验证层面面临资料不对称的挑战。其巨大的单芯片内存、液冷高密度 pod 和 Jupiter 网络，标志着 Google 在 “超算即服务” 市场上的全力竞争。

在软件生态方面，XLA 编译器与 TPU 的协同设计是其实现高能效的关键，但也带来了对动态形状和自定义算子支持不足的问题。专利布局分析揭示了 Google“论文开放、专利收敛” 的策略，以及 AI 加速器竞争正在向编译器-软件栈转移的趋势。

展望未来，TPU 面临的未解张力包括：脉动阵列对稀疏性和动态推理的适应、封闭生态与开放研究之间的平衡、以及厂商宣称与独立验证之间的鸿沟。这些问题不仅关乎 TPU 自身的演进，也将深刻影响整个 AI 加速器领域的发展轨迹。

致谢：本文的撰写基于 Google 发布的核心技术文献（Jouppi et al., ISCA 2017; Jouppi et al., ISCA 2023; Norrie et al., IEEE Micro 2021），以及 Semianalysis 等第三方分析机构的调研报告。本文写作过程中得到了 Claude (Anthropic) 的辅助，包括文献检索、内容整理、技术概念梳理和文本撰写。特此说明。

参考文献

[1] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. “In-Datacenter Performance Analysis of a Tensor Processing Unit.” *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1-12. DOI: 10.1145/3079856.3080246

[2] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, et al. “TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings.” *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1-14. DOI: 10.1145/3579371.3589350

[3] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, et al. “Ten Lessons From Three Generations Shaped Google’s TPUv4i: Industrial Product.” *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1-14. DOI: 10.1109/ISCA52012.2021.00010

[4] Thomas Norrie, Nishant Patil, Doe Hyun Yoon, et al. “The Design Process for Google’s Training Chips: TPUv2 and TPUv3.” *IEEE Micro*, vol. 41, no. 2, 2021, pp. 56-63. DOI: 10.1109/MM.2021.3058217

[5] H. T. Kung. “Why Systolic Architectures?” *IEEE Computer*, vol. 15, no. 1, 1982, pp. 37-46.

[6] David Patterson, Joseph Gonzalez, Quoc Le, et al. “Carbon Emissions and Large Neural Network Training.” arXiv:2104.10350, 2021.

[7] SemiAnalysis. "Google AI Infrastructure Supremacy: Systems Matter More Than Microarchitecture." SemiAnalysis Newsletter, 2025.

[8] Google. "Ironwood: The first Google TPU for the age of inference." Google Blog, 2025. https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/ironwood-tpu-age-of-inference/

[9] Google. "Introducing Trillium: 6th Gen TPUs." Google Blog, 2024.

[10] MLCommons. "MLPerf Training v3.0 Results." 2023. https://mlcommons.org/training-results/